Sprint 7 : JavaScript Loops and Iteration.

මේ අධ්‍යයන සටහන සකස් කරලා තියෙන්නේ JavaScript වල තියෙන Loops සහ Iteration සංකල්ප ඉතාමත් සරලව, පහසුවෙන් තේරුම් ගන්න පුළුවන් වෙන විදිහටයි. programming වලදී පාවිච්චි වෙන English වචන (keywords) ඒ ආකාරයෙන්ම තබාගෙන, එදිනෙදා සිංහල භාෂාවෙන් මේවා පැහැදිලි කරලා තියෙනවා.

Loops and Iteration Introduction

Concept Explanation (Sinhala): Sprint 7 වලදී අපි ප්‍රධාන වශයෙන්ම ඉගෙන ගන්නේ JavaScript වල ආරක්ෂිතව (safely) එකම වැඩක් නැවත නැවත සිදු කරන්නේ (repeating work) කොහොමද කියන එකයි.

කලින් sprints වලදී ඔයා variables හදන්න, values store කරන්න, conditions ලියන්න, functions ලියන්න වගේම Arrays ඇතුළේ values ගොඩක් store කරලා තියාගන්න ඉගෙන ගත්තා. මේ Sprint 7 එකෙන් අපි ඉගෙන ගන්නේ අන්න ඒ Arrays ඇතුළේ තියෙන values එකින් එක (one by one) පරීක්ෂා කරමින් එකම code එක නැවත නැවත run කරන්නේ කොහොමද කියන එකයි.

අපේ ප්‍රධාන ඉලක්කය තමයි වැඩක් ආරක්ෂිතව repeat කරන එක සහ අපේ data වලට ගැලපෙනම loop වර්ගය තෝරාගන්න එක.

The Core Idea of Loops

Concept Explanation (Sinhala): Loop එකකින් කරන්නේ එකම code block එකක් නැවත නැවතත් (repeat) run කරන එකයි.

හිතන්න අපිට Array එකක තියෙන values ටිකක් print කරන්න ඕන කියලා. Loop එකක් නැතුව ඒක කරන්න ගියොත් අපිට හැම index එකකටම වෙන වෙනම `console.log()` ලියන්න වෙනවා. මේක ඉතාමත් කම්මැලි සහ එකම දේ නැවත නැවත ලියන (repetitive) වැඩක්. නමුත් Loop එකක් පාවිච්චි කරලා අපිට මේ වැඩේ ඉතාමත් පිරිසිදුව (clean code එකකින්) කරන්න පුළුවන්.

අපිට Loops ප්‍රධාන වශයෙන්ම වැදගත් වෙන්නේ මේ වගේ වෙලාවල් වලටයි:

- values ගොඩක් එකපාර print කරන්න.
- totals (එකතුවල්) ගණනය කරන්න.
- යම් කිසි item එකක් search (සොයා ගැනීමට) කරන්න.
- අපිට ගැලපෙන values කීයක් තියෙනවද කියලා count කරන්න.
- values ගොඩක් validate (නිවැරදිදැයි පරීක්ෂා) කරන්න.
- values ගොඩක් එකවර update කරන්න.
- grid (කොටු දැලක්) එකක තියෙන හැම row සහ column එකක්ම කියවන්න.

Code Block:

```
// Loop එකක් නැතුව (Repetitive way)
const scores = [80, 95, 72, 100];
console.log(scores[0]);
console.log(scores[1]);
console.log(scores[2]);
console.log(scores[3]);

// Loop එකක් පාවිච්චි කරලා (Cleaner way)
for (const score of scores) {
  console.log(score);
}
```

Code Breakdown (Sinhala):

- පළමු ක්‍රමයේදී අපිට `scores` Array එකේ තියෙන හැම index එකකටම ($0, 1, 2, 3$) වෙන වෙනම `console.log()` ලියන්න වුණා.
- දෙවැනි ක්‍රමයේදී අපි `for...of` කියන loop එක පාවිච්චි කරලා තියෙනවා. ඒකෙන් කියන්නේ: "මේ `scores` Array එකේ තියෙන හැම `score` එකක් සඳහාම (`for each score in scores`), මේ block එක ඇතුළේ තියෙන code එක run කරන්න" කියන එකයි.

Standard for Loops

Concept Explanation (Sinhala): අපිට loop එක run වෙන වාර ගණන පාලනය කරන්න index එකක් (index control) අත්‍යවශ්‍ය වෙන වෙලාවට අපි Standard for loop එකක් පාවිච්චි කරනවා.

Standard for loop එකක ප්‍රධාන කොටස් 3ක් තියෙනවා:

1. **Initialization:** Loop එකේ counter එක පටන් ගන්න අගය (උදාහරණයක් විදිහට `let i = 0` කියන්නේ counter එක 0 න් පටන් ගන්නවා කියන එක).
2. **Condition:** Loop එක දිගටම run වෙන්න බලපාන කොන්දේසිය. මේ condition එක true වෙලා තියෙනකම් විතරයි loop එක run වෙන්නේ.
3. **Update:** Loop එකේ හැම වටයක්ම (cycle) ඉවර වුණාම counter එකට වෙන වෙනස (උදාහරණයක් විදිහට `i++` කියන්නේ counter එක 1 කින් වැඩි කරනවා කියන එක).

Code Block:

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

Code Breakdown (Sinhala):

- `let i = 0` : loop counter එක 0 න් ආරම්භ කරනවා (Initialization).
- `i < 5` : i හි අගය 5 ට වඩා අඩු තාක් කල් loop එක දිගටම run වෙනවා (Condition).
- `i++` : හැම loop cycle එකක් අවසානයේදීම i හි අගයට 1 ක් එකතු වෙනවා (Update).
- මේ අනුව output එක විදිහට $0, 1, 2, 3, 4$ කියන ඉලක්කම් ටික print වෙනවා. i හි අගය 5 වුණු ගමන් condition එක false වෙන නිසා loop එක නතර වෙනවා.

What i Means

Concept Explanation (Sinhala): ඔයා සාමාන්‍යයෙන් loops ලියද්දී හැම තැනකදීම වගේ `i` කියන අකුර දකිනවා ඇති. මේ `i` කියන්නේ **index** එහෙමත් නැත්නම් **counter** කියන එක කෙටි කරලා හදාගත්ත variable name එකක් විතරයි.

සරලවම කිව්වොත්, `i` මඟින් කරන්නේ අපි මේ වෙලාවේ loop එකේ කීවෙනි පියවරේද (step) ඉන්නේ කියලා track කරන එකයි. Arrays සහ Strings වල indexes පටන් ගන්නේ 0 න් නිසා, අපි සාමාන්‍යයෙන් `i` එක 0 න් ආරම්භ කරනවා.

Code Block:

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

Code Breakdown (Sinhala): මෙහිදී `i` යනු loop එක ඇතුළේ පාවිච්චි වන counter variable එකයි. එය 0 සිට 4 දක්වා වෙනස් වෙමින් පියවරෙන් පියවර ඉදිරියට යයි.

How a for Loop Runs Step by Step

Concept Explanation (Sinhala): අපි බලමු Standard for loop එකක් පියවරෙන් පියවර (Dry run) ක්‍රියාත්මක වෙන්නේ කොහොමද කියලා. $i < 3$ කියන condition එක පාවිච්චි කරලා ලියපු loop එකක පියවර මෙන්න මේ විදිහටයි සිද්ධ වෙන්නේ:

Step	i Value	Condition ($i < 3$)	What Happens
Step 1	0	`true`	0 print කරනවා, ඊටපස්සේ `i++` මඟින් i හි අගය 1 බවට පත් වෙනවා.
Step 2	1	`true`	1 print කරනවා, ඊටපස්සේ `i++` මඟින් i හි අගය 2 බවට පත් වෙනවා.
Step 3	2	`true`	2 print කරනවා, ඊටපස්සේ `i++` මඟින් i හි අගය 3 බවට පත් වෙනවා.
Step 4	3	`false`	Condition එක false නිසා loop එක මෙතනින් නතර වෙනවා (Stop).

මතක තියාගන්න, $i < 3$ කියන එකෙන් අදහස් වෙන්නේ i හි අගය 3 වෙන්න කලින් loop එක නතර වන බවයි.

Looping Through an Array With a Standard for Loop

Concept Explanation (Sinhala): අපිට Array එකක තියෙන values ටිකක් print කරද්දී, ඒ values වල index එකත් අවශ්‍ය වෙනවා නම් අපි Standard for loop එකක් පාවිච්චි කරනවා.

මෙහිදී loop condition එක විදිහට අපි දෙන්නේ `i < array.length` කියන එකයි. උදාහරණයක් විදිහට Array එකේ values 3 ක් තියෙනවා නම්, length එක 3 වෙනවා. එනකොට loop එක run වෙන්නේ i හි අගය $0, 1, 2$ වෙද්දී විතරයි. ඒ කියන්නේ හරියටම Array එකේ තියෙන valid indexes ටිකට ගැලපෙන්න loop එක ක්‍රියාත්මක වෙනවා.

Code Block:

```
const fruits = ["apple", "banana", "cherry"];

for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}
```

Code Breakdown (Sinhala):

- `fruits.length` එක 3 නිසා loop එක $i < 3$ වෙනකම් run වෙනවා.
- `i` මඟින් අපිට වත්මන් index එක (0, 1, 2) ලබා දෙනවා.
- `fruits[i]` මඟින් ඒ අදාළ index එකේ තියෙන value එක ("apple", "banana", "cherry") ලබා දෙනවා.
- Output එක විදිහට:

```
0 apple
1 banana
2 cherry
```

Why Loops Often Start at 0

Concept Explanation (Sinhala): Arrays වල පළවෙනි අගය තියෙන්නේ හැමවෙලේම 0 වෙනි index එකේ. ඒ නිසයි අපි Array එකක් loop කරන්න පටන් ගද්දි හැමතිස්සෙම `let i = 0` වලින් පටන් ගන්නේ.

ආරම්භක රීතිය (Beginner Rule): Array එකක් indexes පාවිච්චි කරලා loop කරනවා නම්, හැමවෙලේම loop එක 0 න් ආරම්භ කරලා, `i < array.length` කියන condition එක පාවිච්චි කරන්න.

Off-by-One Mistakes

Concept Explanation (Sinhala): **Off-by-One mistake** එකක් කියන්නේ අපි ලියන condition එකේ පොඩි වැරැද්දක් නිසා loop එක අපිට ඕන වාර ගණනට වඩා එකක් වැඩියෙන් හෝ අඩුවෙන් run වීමයි.

ආධුනිකයන් අතින් සිද්ධ වෙන ලොකුම වැරැද්දක් තමයි Array loop එකක condition එකට `i <= array.length` කියලා දැමීම. Array එකක values 3 ක් තිබ්බොත්, අන්තිම value එකේ index එක වෙන්නේ 2. හැබැයි අපි `i <= 3` දැම්මොත්, loop එක index 3 වෙනකම්ම run වෙන්න උත්සාහ කරනවා. Index 3 හි කිසිම value එකක් නැති නිසා අපිට `undefined` කියන එක output එක විදිහට ලැබෙනවා.

Code Block:

```
// වැරදි ක්‍රමය (Problematic example)
const fruits = ["apple", "banana", "cherry"];
for (let i = 0; i <= fruits.length; i++) {
  console.log(fruits[i]);
}

// නිවැරදි ක්‍රමය (Correct version)
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Code Breakdown (Sinhala):

- පළමු වැරදි ක්‍රමයේදී output එකේ අන්තිමට `undefined` print වෙනවා. මොකද `fruits[3]` කියලා අගයක් නැති නිසා.
- දෙවැනි නිවැරදි ක්‍රමයේදී `i < fruits.length` (i < 3) පාවිච්චි කර ඇති නිසා loop එක හරියටම index 2 න් නතර වෙනවා.

for...of

Concept Explanation (Sinhala): ඔයාට Array එකක තියෙන values ටික කෙලින්ම (directly) ලබාගන්න විතරක් අවශ්‍ය නම් සහ indexes පාවිච්චි කරන්න අවශ්‍යතාවයක් නැත්නම්, පාවිච්චි කරන්න පුළුවන් ලස්සනම සහ පිරිසිදුම ක්‍රමය තමයි `for...of` loop එක.

මේක කියවන්නේ මෙහෙමයි: "grades Array එකේ තියෙන හැම grade එකක් සඳහාම, මේ code එක run කරන්න."

Code Block:

```
const grades = [90, 82, 77];

for (const grade of grades) {
  console.log(grade);
}
```

Code Breakdown (Sinhala): මෙහිදී index එකක් නැතුවම, Array එකේ තියෙන values ($90, 82, 77$) එකින් එක කෙලින්ම `grade` කියන variable එකට ලැබෙනවා.

for...of With Strings

Concept Explanation (Sinhala): JavaScript වල Strings කියන්නේත් iterable (එකින් එක කියවිය හැකි) දේවල් වලටයි. ඒ නිසා අපිට `for...of` පාවිච්චි කරලා String එකක තියෙන අකුරු (characters) එකින් එක වෙන් කරලා ලූප් කරන්නත් පුළුවන්.

Code Block:

```
const word = "code";

for (const letter of word) {
  console.log(letter);
}
```

Code Breakdown (Sinhala):

- `"code"` කියන String එකේ තියෙන හැම අකුරක්ම (`c` , `o` , `d` , `e`) එකින් එක වෙන් වෙලා print වෙනවා.

When to Use for...of

Concept Explanation (Sinhala): අපි `for...of` පාවිච්චි කරන්නේ මේ වගේ අවස්ථාවලදීයි:

- Array එකක් loop කිරීමට.
- String එකක් loop කිරීමට.
- අපිට අවශ්‍ය වන්නේ එහි තියෙන values පමණක් වන විට.
- අපිට index එකක් අවශ්‍ය නොවන විට.
- අපිට code එක ඉතාමත් සරලව සහ පිරිසිදුව ලියන්න අවශ්‍ය වූ විට.

ආරම්භක රීතිය (Beginner Rule): Array එකක හෝ String එකක values ටික විතරක් ගන්න ඕන නම් හැමවෙලේම `for...of` පාවිච්චි කරන්න.

Standard for Loop vs for...of

Concept Explanation (Sinhala): මේ loop වර්ග දෙකම Array එකක් loop කරන්න පාවිච්චි කරන්න පුළුවන්. නමුත් අපේ අවශ්‍යතාවය අනුව නිවැරදි එක තෝරාගන්න ඕනේ:

Situation (අවස්ථාව)	Better Choice (වඩාත්ම ගැලපෙන තේරීම)
ඔයාට index එක අත්‍යවශ්‍යම නම්	Standard `for` loop
ඔයාට value එක විතරක් ප්‍රමාණවත් නම්	`for...of`
Index එකේ පිහිටීම අනුව Array එක update කරන්න ඕන නම්	Standard `for` loop
නිකන්ම values කියවන්න (read) විතරක් ඕන නම්	`for...of`

Code Block:

```
const colors = ["red", "green", "blue"];

// Standard for loop (Index එකත් සමඟ)
for (let i = 0; i < colors.length; i++) {
  console.log(i, colors[i]);
}

// for...of loop (Value එක පමණක්)
for (const color of colors) {
  console.log(color);
}
```

for...in

Concept Explanation (Sinhala): අපිට Object එකක් ඇතුළේ තියෙන properties (Keys) ටික loop කරන්න අවශ්‍ය වුණාම පාවිච්චි කරන්නේ `for...in` loop එකයි.

මතක තියාගන්න, `for...in` එකෙන් අපිට කෙලින්ම ලැබෙන්නේ Object එකේ තියෙන **keys** (properties) ටිකයි.

Code Block:

```
const user = {
  name: "Ada",
  active: true,
  score: 95
};

for (const key in user) {
  console.log(key);
}
```

Code Breakdown (Sinhala):

- Output එක විදිහට ලැබෙන්නේ Object එකේ keys ටිකයි:

```
name
active
score
```

Getting Object Values With for...in

Concept Explanation (Sinhala): අපි කලින් දැක්කා `for...in` වලින් ලැබෙන්නේ keys ටික විතරයි කියලා. එනකොට අපිට ඒ keys වලට අදාළ values ටික ගන්න ඕන නම් මොකද කරන්නේ?

අන්න ඒ වෙලාවට අපි **bracket notation** (`object[key]`) පාවිච්චි කරනවා. මෙතනදී `user.key` කියලා ලියන්න බැහැ, මොකද `key` කියන්නේ dynamic variable එකක් නිසා.

Code Block:

```
const user = {
  name: "Ada",
  active: true,
  score: 95
};

for (const key in user) {
  console.log(key, user[key]);
}
```

Code Breakdown (Sinhala):

- `key` කියන variable එක ඇතුළට පිළිවෙලින් `"name"` , `"active"` , `"score"` කියන keys ටික එනවා.
- `user[key]` මඟින් dynamic ලෙස ඒ ඒ key එකට අදාළ value එක (`"Ada"` , `true` , `95`) access කරනවා.
- Output එක:

```
name Ada
active true
score 95
```

for...of vs for...in

Concept Explanation (Sinhala): මේ loops දෙක පැටලෙන්න තියෙන ඉඩකඩ වැඩියි. ඒ නිසා මේ වෙනස හොඳින් මතක තියාගන්න:

Loop	Main Use (ප්‍රධාන භාවිතය)	What It Gives You (ලබා දෙන දේ)
`for...of`	Arrays සහ Strings සඳහා	Values (අගයන්)
`for...in`	Objects සඳහා	Keys (ගුණාංග වල නම්)

Code Block:

```
const fruits = ["apple", "banana"];
// for...of පාවිච්චි කරලා values ගන්නවා
for (const fruit of fruits) {
  console.log(fruit); // Output: apple, banana
}

const user = { name: "Ada", active: true };
// for...in පාවිච්චි කරලා keys ගන්නවා
for (const key in user) {
  console.log(key, user[key]); // Output: name Ada, active true
}
```

Avoid Using for...in as Your Default Array Loop

Concept Explanation (Sinhala): තාක්ෂණිකව බැලුවොත් Array එකක් loop කරන්නත් `for...in` පාවිච්චි කරන්න පුළුවන් (මොකද Array එකක් කියන්නෙත් Object එකක් නිසා). හැබැයි ආධුනිකයෙක් විදිහට Array එකක් loop කරන්න කවදාවත් `for...in` පාවිච්චි කරන්න එපා.

මොකද `for...in` මඟින් Array එකක තියෙන indexes ලබා දෙන්නේ number values විදිහට නෙවෙයි, string values (උදාහරණයක් විදිහට `"0"` , `"1"`) විදිහටයි. ඒ නිසා ගණිතමය වැඩක් කරන්න ගියොත් code එක වරද්දින්න පුළුවන්.

ආරම්භක රීතිය (Beginner Rule): Array එකක් සඳහා හැමවෙලේම `for...of` (values සඳහා) හෝ Standard `for` loop (indexes සඳහා) පාවිච්චි කරන්න. `for...in` පාවිච්චි කරන්න එපා.

while Loops

Concept Explanation (Sinhala): `while` loop එකක් ක්‍රියාත්මක වෙන්නේ අපි ලබා දෙන condition එක true වෙලා තියෙන තාක් කල් විතරයි. හැම loop cycle එකක්ම පටන් ගන්න කලින් condition එක true ද කියලා check කරනවා.

Code Block:

```
let count = 3;

while (count > 0) {
  console.log(count);
  count--;
}
```

Code Breakdown (Sinhala):

- ආරම්භයේදී `count` එක 3 යි. $3 > 0$ කියන condition එක true නිසා loop එක ඇතුළට යනවා.
- 3 print කරලා `count--` මඟින් `count` එක 2 කරනවා.
- ඊළඟ cycle එකේදී $2 > 0$ true නිසා 2 print කරලා `count` එක 1 කරනවා.
- ඊළඟට $1 > 0$ true නිසා 1 print කරලා `count` එක 0 කරනවා.
- දැන් $0 > 0$ කියන condition එක `false` වෙනවා. ඒ නිසා loop එක නතර වෙනවා.
- Output එක: 3, 2, 1

When to Use while

Concept Explanation (Sinhala): අපිට loop එක හරියටම කී වතාවක් run කරන්න ඕනද කියලා කලින් දන්නේ නැති වෙලාවල් වලදී, හැබැයි loop එක නතර කරන්න ඕන condition එක විතරක් දන්න වෙලාවල් වලදී අපි `while` loop එකක් පාවිච්චි කරනවා.

උදාහරණයක් විදිහට:

- යම් කිසි limit එකක් වෙනකම් value එකක් වැඩි/අඩු කරන්න ඕන වුණාම.
- යම් කොන්දේසියක් වෙනස් වන තුරු දිගටම run කරන්න අවශ්‍ය වුණාම.

Code Block:

```
let energy = 5;

while (energy > 0) {
  console.log("Keep working");
  energy--;
}
```

Infinite Loops

Concept Explanation (Sinhala): **Infinite loop** එකක් කියන්නේ කවදාවත් නතර වෙන්නේ නැතිව දිගටම run වෙන loop එකකටයි. මේක ඉතාමත් භයානකයි, මොකද මේකෙන් ඔයාගේ web browser එක හෝ computer එක freeze වෙන්න (හැන්ග් වෙන්න) පුළුවන්.

මෙහෙම වෙන්නේ loop condition එක කවදාවත් false නොවන විදිහට code එක ලිව්වොත් විතරයි. උදාහරණයක් විදිහට පහත වැරදි code එකේ `count` එක වෙනස් වෙන්නේ නෑ. ඒ නිසා `count > 0` කියන condition එක හැමදාටම true වෙනවා.

Code Block:

```
// වැරදි ක්‍රමය - Infinite Loop (Dangerous)
let count = 3;
while (count > 0) {
  console.log(count);
  // මෙතන count එක අඩු කරන්නේ නෑ!
}

// නිවැරදි ක්‍රමය (Safe)
let count2 = 3;
while (count2 > 0) {
  console.log(count2);
  count2--; // අගය වෙනස් කරනවා
}
```

ආරම්භක රීතිය (Beginner Rule): `while` loop එකක් ලියද්දී, condition එකට බලපාන variable එක loop එක ඇතුළේදී අනිවාර්යයෙන්ම වෙනස් (update) වෙනවා කියලා තහවුරු කරගන්න.

Infinite Loop Safety Checklist

Concept Explanation (Sinhala): `while` loop එකක් run කරන්න කලින් මේ දේවල් 5 පරීක්ෂා කරලා බලන්න:

1. Loop එක පාලනය කරන condition එක මොකක්ද?
2. ඒ condition එක රඳා පවතින්නේ මොන variable එක මතද?
3. ඒ variable එක loop එක ඇතුළේදී වෙනස් වෙනවද?
4. Condition එක කවදා හරි දවසක `false` බවට පත් වෙනවද?
5. Loop එක නතර වෙන්න පැහැදිලි අවසානයක් තියෙනවද?

do...while Loops

Concept Explanation (Sinhala): `do...while` loop එකක් සාමාන්‍ය `while` loop එකකට වඩා පොඩ්ඩක් වෙනස්. මේකේදී මුලින්ම code block එක run කරලා, ඊට පස්සේ තමයි condition එක check කරන්නේ.

ඒ නිසා මතක තියාගන්න, `do...while` loop එකක් condition එක true වුණත් නැතත් අඩුම තරමේ එක පාරක්වත් (at least once) අනිවාර්යයෙන්ම run වෙනවා.

Code Block:

```
let count = 0;

do {
  console.log("Runs once");
  count++;
} while (count < 1);
```

Code Breakdown (Sinhala): මුලින්ම condition එක බලන්නේ නැතුව `"Runs once"` කියන එක print කරලා `count` එක 1 කින් වැඩි කරනවා. ඊට පස්සේ තමයි `count < 1` ($1 < 1$) ද කියලා බලන්නේ. එය false නිසා loop එක නතර වෙනවා.

while vs do...while

Concept Explanation (Sinhala): මේ දෙකේ ප්‍රධාන වෙනස තියෙන්නේ condition එක පරීක්ෂා කරන වෙලාව අනුවයි:

Loop	Checks Condition (Condition එක බලන්නේ)	Can Run Zero Times? (0 වතාවක් run වෙන්න පුළුවන්ද?)	Always Runs At Least Once? (අඩුම තරමේ එක පාරක්වත් run වෙනවද?)
`while`	Code එක run වෙන්න කලින්	**Yes** (මුලින්ම false වුණොත් එක පාරක්වත් run වෙන්නේ නෑ)	**No**
`do...while`	Code එක run වුණාට පස්සේ	**No**	**Yes**

Code Block:

```
let number = 10;

// while loop එක (එකපාරක්වත් run වෙන්නේ නැහැ)
while (number < 5) {
  console.log("while ran");
}

// do...while loop එක (එක පාරක් run වේ)
do {
  console.log("do...while ran");
} while (number < 5);
```

break

Concept Explanation (Sinhala): `break` keyword එක පාවිච්චි කරන්නේ loop එකක් සාමාන්‍ය විදිහට ඉවර වෙන්න කලින්, අපිට අවශ්‍ය වෙලාවක ඒකෙන් එකපාරටම එළියට යන්න (exit early) අවශ්‍ය වුණාමයි.

උදාහරණයක් විදිහට, අපි Array එකක යම් අගයක් සොයනවා නම් (searching), ඒ අගය හමු වූ විහාම තවදුරටත් loop එක run කිරීම තේරුමක් නෑ (වැඩ කන්දරාවක් අපතේ යාමකි). අන්න ඒ වෙලාවට අපි `break` පාවිච්චි කරලා loop එක නතර කරනවා.

Code Block:

```
const scores = [40, 55, 72, 90];

for (const score of scores) {
  if (score >= 70) {
    console.log(`First passing score: ${score}`);
    break;
  }
}
```

Code Breakdown (Sinhala):

- Loop එක 40 සහ 55 පරීක්ෂා කරලා ඒවා 70 ට වඩා අඩු නිසා ඉදිරියට යනවා.
- 72 හමු වූ විට, condition එක true වෙනවා. එතකොට `"First passing score: 72"` print කරලා `break` වෙනවා.
- `break` නිසා loop එක එතනින්ම නතර වන අතර, ඊළඟට තියෙන 90 පරීක්ෂා කරන්න යන්නේ නැත.

continue

Concept Explanation (Sinhala): `continue` keyword එකෙන් කරන්නේ loop එකේ වත්මන් පියවර (current step) පමණක් මඟ හැරලා (skip කරලා), කෙලින්ම ඊළඟ පියවරට (next iteration) මාරු වෙන එකයි.

ඒ කියන්නේ loop එක සම්පූර්ණයෙන්ම නතර වෙන්නේ නෑ, අදාළ අගය පමණක් skip කරලා ඉතිරි ටික සාමාන්‍ය පරිදි run වෙනවා.

Code Block:

```
const scores = [80, 95, 72, 100];
let total = 0;

for (const score of scores) {
  if (score < 75) {
    continue;
  }
  total += score;
}
console.log(total);
```

Code Breakdown (Sinhala):

- 80 සහ 95 සාමාන්‍ය පරිදි `total` එකට එකතු වෙනවා.
- 72 හමු වූ විට `score < 75` true වන නිසා `continue` ක්‍රියාත්මක වෙනවා. ඒ නිසා 72 එකතු කරන්නේ නැතුව skip කරලා කෙලින්ම ඊළඟ අගය වන 100 වෙත මාරු වෙනවා.
- Output එක: 275 $(80 + 95 + 100)$.

break vs continue

Concept Explanation (Sinhala): මේ දෙකම loop එකේ ගලායාම (control flow) පාලනය කරන්න පාවිච්චි කරත්, කරන්නේ හාත්පසින්ම වෙනස් වැඩ දෙකක්:

Keyword	Meaning (තේරුම)	What Happens Next? (ඊළඟට වෙන්නේ මොකක්ද?)
`break`	මුළු loop එකම වහාම නතර කරන්න.	Loop එකෙන් සම්පූර්ණයෙන්ම එළියට යනවා.
`continue`	මේ වටය (step) විතරක් skip කරන්න.	ඊළඟ වටයට (iteration) මාරු වී loop එක දිගටම කරගෙන යනවා.

Code Block:

```
const numbers = [1, 2, 3, 4];

// break භාවිතය
for (const number of numbers) {
  if (number === 3) break;
  console.log(number); // Output: 1, 2
}

// continue භාවිතය
for (const number of numbers) {
  if (number === 3) continue;
  console.log(number); // Output: 1, 2, 4
}
```

Conditions Inside Loops

Concept Explanation (Sinhala): Loops ඇතුළේදී අපි නිතරම `if` statements පාවිච්චි කරනවා. මෙයින් කරන්නේ loop එක හරහා යන හැම අගයක්ම එකින් එක පරීක්ෂා කරලා, කොන්දේසියට ගැලපෙනවා නම් පමණක් යම් ක්‍රියාවක් සිදු කිරීමයි.

Code Block:

```
const scores = [90, 45, 80, 60];

for (const score of scores) {
  if (score >= 70) {
    console.log(`${score} is passing`);
  } else {
    console.log(`${score} is not passing`);
  }
}
```

Accumulators in Loops

Concept Explanation (Sinhala): **Accumulator** එකක් කියන්නේ loop එක දිගේ අනුපිළිවෙලින් යම්කිසි ප්‍රතිඵලයක් එකතු කරලා තියාගන්න පාවිච්චි කරන variable එකකටයි (උදාහරණයක් විදිහට මුළු එකතුව - sum එකක් එකතු කරන්න).

අපි සාමාන්‍යයෙන් පාවිච්චි කරන accumulator patterns කිහිපයක් මෙන්න:

- `let total = 0;` (එකතුවල් සඳහා)
- `let count = 0;` (ගණන් කිරීම් සඳහා)
- `let result = "";` (අලුත් string එකක් සාදා ගැනීමට)
- `let found = false;` (යමක් තිබේදැයි සෙවීම සඳහා)

Code Block:

```
const prices = [10, 20, 30];
let total = 0; // Accumulator variable එකක්

for (const price of prices) {
  total += price;
}
console.log(total); // Output: 60
```

Counting With Loops

Concept Explanation (Sinhala): අපිට Array එකක තියෙන values අතරින් අපේ කොන්දේසියට ගැලපෙන values කීයක් තියෙනවද කියලා ගණන් කරන්න (count කිරීමට) accumulator pattern එකක් පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
const scores = [90, 45, 80, 60, 100];
let passingCount = 0;

for (const score of scores) {
  if (score >= 70) {
    passingCount++;
  }
}
console.log(passingCount); // Output: 3
```

Code Breakdown (Sinhala):

- loop එක ක්‍රියාත්මක වෙද්දී 70 ට වඩා වැඩි හෝ සමාන අගයන් (90, 80, 100) හමුවන වාරයක් පාසා `passingCount` එක 1 කින් වැඩි කර ගන්නවා.

Searching With Loops

Concept Explanation (Sinhala): අපිට Array එකක යම්කිසි අගයක් තියෙනවද කියලා සොයන්න (search කිරීමට) accumulator එකක් සහ `break` එකක් එකතු කරලා ලස්සන code එකක් ලියන්න පුළුවන්.

Code Block:

```
const names = ["Ada", "Grace", "Linus"];
let found = false;

for (const name of names) {
  if (name === "Grace") {
    found = true;
    break;
  }
}
console.log(found); // Output: true
```

Building a New Array With a Loop

Concept Explanation (Sinhala): අපිට loop එකක් පාවිච්චි කරලා පවතින Array එකකින් තෝරාගත් අගයන් පමණක් අලුත් Array එකකට ඇතුළත් කරලා අලුත්ම Array එකක් නිර්මාණය කරන්න පුළුවන්. මේ සඳහා අපි මුලින්ම හිස් Array එකක් (`[]`) හදාගෙන, ඊට පස්සේ `.push()` method එක පාවිච්චි කරනවා.

Code Block:

```
const scores = [90, 45, 80, 60, 100];
const passingScores = []; // හිස් Array එකක්

for (const score of scores) {
  if (score >= 70) {
    passingScores.push(score);
  }
}
console.log(passingScores); // Output: [90, 80, 100]
```

Nested Loops

Concept Explanation (Sinhala): **Nested Loop** එකක් කියන්නේ loop එකක් ඇතුළේ තවත් loop එකක් ලියා තිබීමයි.

මෙහිදී ක්‍රියාවලිය සිද්ධ වෙන්නේ මෙහෙමයි: පිටතින් තියෙන loop එකේ (outer loop) එක පියවරක් සඳහා, ඇතුළතින් තියෙන loop එක (inner loop) **සම්පූර්ණයෙන්ම** run වෙලා ඉවර වෙනවා.

Code Block:

```
for (let outer = 1; outer <= 2; outer++) {
  for (let inner = 1; inner <= 3; inner++) {
    console.log(outer, inner);
  }
}
```

Code Breakdown (Sinhala):

- `outer` එක 1 වෙද්දී, `inner` එක 1, 2, 3 දක්වා සම්පූර්ණයෙන්ම ලූප් වෙනවා.
- `outer` එක 2 වෙද්දී, නැවතත් `inner` එක 1, 2, 3 දක්වා සම්පූර්ණයෙන්ම ලූප් වෙනවා.
- Output එක:

```
1 1
1 2
1 3
2 1
2 2
2 3
```

Nested Loops With 2D Arrays

Concept Explanation (Sinhala): **2D Array** එකක් කියන්නේ Array එකක් ඇතුළේ තවත් Arrays තියෙන grid (කොටු දැලක්) වගේ ව්‍යුහයකටයි. මෙවැනි 2D Array එකක තියෙන හැම value එකක්ම කියවන්න (read කිරීමට) අපිට අනිවාර්යයෙන්ම nested loop එකක් පාවිච්චි කරන්න වෙනවා.

පිටත loop එකෙන් row (පේළි) එකින් එක කියවන අතර, ඇතුළත loop එකෙන් ඒ row එක ඇතුළේ තියෙන col (column/තීරු) එකින් එක කියවනවා.

Code Block:

```
const grid = [
  ["A1", "A2"],
  ["B1", "B2"]
];

for (let row = 0; row < grid.length; row++) {
  for (let col = 0; col < grid[row].length; col++) {
    console.log(row, col, grid[row][col]);
  }
}
```

Code Breakdown (Sinhala):

- `grid[row][col]` මඟින් අදාළ පේළියේ (row) සහ තීරුවේ (column) පිහිටි value එක ලබා දෙනවා.
- Output එක:

```
0 0 A1
0 1 A2
1 0 B1
1 1 B2
```

Nested Loops Can Become Slow

Concept Explanation (Sinhala): Nested loops ඉතාමත් ප්‍රයෝජනවත් වුණත්, විශාල දත්ත ප්‍රමාණයක් (large data) සමඟ වැඩ කරද්දී මේවා සෑහෙන්න slow වෙන්න පුළුවන්.

උදාහරණයක් විදිහට:

- listA එකේ items 3 ක් සහ listB එකේ items 4 ක් තිබුණොත් ඇතුළත code එක run වෙන වාර ගණන $3 \times 4 = 12$ යි.
- හැබැයි list දෙකේම items 1000 බැගින් තිබුණොත්, ඇතුළත code එක $1000 \times 1000 = 1,000,000$ (ලක්ෂ 10ක්) වතාවක් run වෙනවා!

ආරම්භක රීතිය (Beginner Rule): Grids හෝ 2D arrays සඳහා nested loops සාමාන්‍ය දෙයක් වුණත්, විශාල දත්ත ප්‍රමාණයක් සමඟ වැඩ කරද්දී අනවශ්‍ය ලෙස nested loops පාවිච්චි කිරීමෙන් වළකින්න.

Loop Choice Guide

Concept Explanation (Sinhala): වැඩක් කරන්න ලූප් එකක් තෝරාගද්දී පහත වගුව පාවිච්චි කරන්න:

Situation (අවස්ථාව)	Best Choice (හොඳම තේරීම)
ඔයාට index numbers අවශ්‍ය නම්	Standard `for` loop
ඔයාට Array එකක values විතරක් අවශ්‍ය නම්	`for...of`
String එකක අකුරු ලූප් කිරීමට	`for...of`
Object එකක keys ලූප් කිරීමට	`for...in`
Object එකක keys වගේම values bracket notation එකෙන් ගන්න	`for...in` සමඟ `object[key]`
කොන්දේසියක් true වෙලා තියෙනකම් ලූප් කරන්න	`while`
අඩුම තරමේ එක පාරක්වත් ලූප් එක run වෙන්නම ඕන නම්	`do...while`
උත්තරය හමු වූ විහාම ලූප් එක නතර කරන්න	`break`
එක් අගයක් පමණක් මඟ හැර ඉදිරියට යන්න	`continue`
Rows සහ Columns (2D Arrays) සඳහා	Nested loops

Core Sprint 7 Code Pattern

Concept Explanation (Sinhala): මෙන්න මේ code pattern එක හොඳින් අධ්‍යයනය කරන්න. මේක ඇතුළේ අපි ඉගෙන ගත්තු ප්‍රධාන ලූප් වර්ග තුනම අඩංගු වෙනවා.

Code Block:

```
const grades = [90, 82, 77];
for (const grade of grades) {
  console.log(grade);
}

const user = { name: "Ada", active: true };
for (const key in user) {
  console.log(key, user[key]);
}

let count = 3;
while (count > 0) {
  console.log(count);
  count--;
}
```

Mini-Build 1: Count Even Numbers

Concept Explanation (Sinhala): Array එකක් ඇතුළේ තියෙන ඉරට්ටේ සංඛ්‍යා (Even numbers) ගණන සෙවීම සඳහා ලියන ලද function එකක් පහත දැක්වේ.

Code Block:

```
function countEvenNumbers(numbers) {
  let count = 0;
  for (const number of numbers) {
    if (number % 2 === 0) {
      count++;
    }
  }
  return count;
}

console.log(countEvenNumbers([1, 2, 3, 4, 5, 6])); // Output: 3
```

Code Breakdown (Sinhala):

- `number % 2 === 0` මඟින් අදාළ අංකය 2 න් බෙදූ විට ඉතිරිය 0 ද (එනම් ඉරට්ටේ සංඛ්‍යාවක්ද) කියා පරීක්ෂා කරයි.
- ඉරට්ටේ සංඛ්‍යා වන 2, 4, 6 හමු වූ විට `count` එක වැඩි වන නිසා අවසාන ප්‍රතිඵලය 3 ලෙස ලැබේ.

Mini-Build 2: Sum Scores 75 and Above

Concept Explanation (Sinhala): Array එකක ඇති ලකුණු අතරින් 75 හෝ ඊට වැඩි ලකුණු පමණක් එකතු කර මුළු එකතුව ලබා දෙන function එකක් මෙහි දැක්වේ. 75 ට අඩු ලකුණු skip කිරීම සඳහා මෙහි `continue` පාවිච්චි කර ඇත.

Code Block:

```
function sumStrongScores(scores) {
  let total = 0;
  for (const score of scores) {
    if (score < 75) {
      continue;
    }
    total += score;
  }
  return total;
}

console.log(sumStrongScores([80, 95, 72, 100])); // Output: 275
```

Code Breakdown (Sinhala): 72 කියන අගය 75 ට වඩා අඩු නිසා `continue` මඟින් එය මඟ හරින අතර, ඉතිරි අගයන් (80, 95, 100) එකතු වී මුළු එකතුව 275 ලෙස ලැබේ.

Mini-Build 3: Find First Passing Score

Concept Explanation (Sinhala): Array එකක ඇති ලකුණු අතරින් 70 හෝ ඊට වැඩි ප්‍රථම ලකුණ සොයා දෙන function එකක් මෙහි දැක්වේ. Function එකක් ඇතුළේදී `return` එකක් ක්‍රියාත්මක වූ විහාම මුළු function එකම එතනින් නතර වේ. මෙය `break` එකක් වැනිය.

Code Block:

```
function findFirstPassingScore(scores) {
  for (const score of scores) {
    if (score >= 70) {
      return score;
    }
  }
  return null;
}

console.log(findFirstPassingScore([40, 55, 72, 90])); // Output: 72
console.log(findFirstPassingScore([40, 55, 60]));     // Output: null
```

Mini-Build 4: Print a Grid

Concept Explanation (Sinhala): Nested loops පාවිච්චි කරමින් 2D Array එකක (Grid) ඇති සියලුම අගයන් තනි තනියම print කරන ආකාරය මෙහි දැක්වේ.

Code Block:

```
function printGrid(grid) {
  for (let row = 0; row < grid.length; row++) {
    for (let col = 0; col < grid[row].length; col++) {
      console.log(grid[row][col]);
    }
  }
}

printGrid([
  ["A1", "A2"],
  ["B1", "B2"],
  ["C1", "C2"]
]);
```

Practice Drill 1: Print Indexes

Concept Explanation (Sinhala): Standard for loop එකක් භාවිතයෙන් 0 සිට 4 දක්වා වූ ලූප් counters (indexes) print කරන ආකාරය.

Code Block:

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

Practice Drill 2: Print Array Values

Concept Explanation (Sinhala): `for...of` loop එකක් භාවිතයෙන් Array එකක ඇති values සරලවම print කරන ආකාරය.

Code Block:

```
const colors = ["red", "green", "blue"];

for (const color of colors) {
  console.log(color);
}
```

Practice Drill 3: Print Object Keys and Values

Concept Explanation (Sinhala): `for...in` loop එකක් සහ bracket notation භාවිතයෙන් Object එකක ඇති keys සහ values ලබා ගන්නා ආකාරය.

Code Block:

```
const course = {
  name: "JavaScript",
  sprint: 7,
  topic: "Loops"
};

for (const key in course) {
  console.log(key, course[key]);
}
```

Practice Drill 4: Fix the Infinite Loop

Concept Explanation (Sinhala): අගය වෙනස් නොකිරීම නිසා ඇතිවන Infinite loop එකක් නිවැරදිව සකස් කරන ආකාරය.

Code Block:

```
// වැරදි ක්‍රමය (Infinite Loop)
/*
let count = 3;
while (count > 0) {
  console.log(count);
}
*/

// නිවැරදි ක්‍රමය (Fixed)
let count = 3;
while (count > 0) {
  console.log(count);
  count--;
}
```

Practice Drill 5: Skip Small Numbers

Concept Explanation (Sinhala): `continue` භාවිතයෙන් 5 ට වඩා අඩු සංඛ්‍යා මඟ හරිමින් (skip කරමින්) අනෙක් සංඛ්‍යා print කරන ආකාරය.

Code Block:

```
const numbers = [2, 5, 8, 1, 10];

for (const number of numbers) {
  if (number < 5) {
    continue;
  }
  console.log(number);
}
```

Practice Drill 6: Stop When You Find a Match

Concept Explanation (Sinhala): `break` භාවිතයෙන් Array එකක් පරීක්ෂා කරගෙන යද්දී `"target"` කියන අගය හමු වූ විහාම ලූප් එක නතර කරන ආකාරය.

Code Block:

```
const words = ["start", "middle", "target", "end"];

for (const word of words) {
  if (word === "target") {
    console.log("Found target");
    break;
  }
  console.log(word);
}
```

Common Beginner Mistakes

Concept Explanation (Sinhala): ආධුනිකයන් නිතරම සිදු කරන ප්‍රධාන වැරදි 5ක් සහ ඒවා වළක්වා ගන්නා ආකාරය මෙන්න:

1. **වැරැද්ද 1: `i <= array.length` භාවිතය.**
   - වැරදි ක්‍රමය: `for (let i = 0; i <= items.length; i++)` -> අන්තිමට `undefined` ලැබේ.
   - නිවැරදි ක්‍රමය: `for (let i = 0; i < items.length; i++)`
2. **වැරැද්ද 2: `while` loop එකක් update කිරීමට අමතක වීම.**
   - loop counter එක වෙනස් නොකළහොත් infinite loop එකක් සෑදේ.
3. **වැරැද්ද 3: `for...of` සහ `for...in` පටලවා ගැනීම.**
   - Array values සඳහා `for...in` පාවිච්චි කළහොත් ලැබෙන්නේ string indexes මිස values නොවේ.
4. **වැරැද්ද 4: Object keys සමඟ bracket notation අමතක වීම.**
   - වැරදි ක්‍රමය: `user.key` (මෙයින් සොයන්නේ "key" යනුවෙන් සැබවින්ම ඇති property එකකි).
   - නිවැරදි ක්‍රමය: `user[key]`
5. **වැරැද්ද 5: `continue` වෙනුවට `break` පාවිච්චි කිරීම.**
   - එක් අයිතමයක් මඟ හැරීමට නම් `continue` ද, මුළු ලූප් එකම නතර කිරීමට `break` ද පාවිච්චි කරන්න.

Beginner Rules for Sprint 7

Concept Explanation (Sinhala): Sprint 7 වලදී මතක තබා ගත යුතු රන්වන් රීති මෙන්න:

1. Indexes අවශ්‍ය නම් Standard `for` loop පාවිච්චි කරන්න.
2. Array/String values පමණක් අවශ්‍ය නම් `for...of` පාවිච්චි කරන්න.
3. Object keys සඳහා `for...in` පාවිච්චි කරන්න.
4. කොන්දේසියක් මත දිවෙන ලූප් සඳහා `while` පාවිච්චි කරන්න.
5. අඩුම තරමේ එක වතාවක්වත් run විය යුතු නම් `do...while` පාවිච්චි කරන්න.
6. ලූප් එකක් කලින් නතර කිරීමට `break` පාවිච්චි කරන්න.
7. එක් පියවරක් මඟ හැරීමට `continue` පාවිච්චි කරන්න.
8. Infinite loops ගැන හැමවිටම අවධානයෙන් ඉන්න.
9. සාමාන්‍ය Array loops සඳහා `i < array.length` පාවිච්චි කරන්න (කිසිවිටක `<=` නොවේ).
10. Grids සහ 2D Arrays සඳහා nested loops පාවිච්චි කරන්න.

Sprint 7 Revision Questions (ස්වයං ඇගයීම් ප්‍රශ්න)

Concept Explanation (Sinhala): නෝට් එක දෙස නොබලා මේ ප්‍රශ්න වලට සිංහලෙන් පිළිතුරු දීමට උත්සාහ කරන්න:

1. Loop එකක් කියන්නේ සරලවම මොකක්ද?
2. Programming වලදී loops වැදගත් වෙන්නේ ඇයි?
3. Standard for loop එකක තියෙන ප්‍රධාන කොටස් 3 මොනවාද?
4. Array loops සාමාන්‍යයෙන් 0 න් ආරම්භ වෙන්නේ ඇයි?
5. `i++` කියන එකෙන් වෙන්නේ මොකක්ද?
6. Array loops වල condition එකට සාමාන්‍යයෙන් $i < array.length$ පාවිච්චි කරන්නේ ඇයි?
7. `for...of` loop එකෙන් අපිට ලැබෙන්නේ මොනවාද?
8. `for...in` loop එකෙන් අපිට ලැබෙන්නේ මොනවාද?
9. `for...in` ලූප් එකක් ඇතුළේදී bracket notation පාවිච්චි කරන්නේ ඇයි?
10. `while` loop එකක් කියන්නේ මොකක්ද?
11. `while` loop එකක් infinite වෙන්න බලපාන හේතුව මොකක්ද?
12. `do...while` loop එකක් සාමාන්‍ය `while` loop එකකින් වෙනස් වෙන්නේ කොහොමද?
13. `break` keyword එකෙන් කරන්නේ මොකක්ද?
14. `continue` keyword එකෙන් කරන්නේ මොකක්ද?
15. Nested loop එකක් කියන්නේ මොකක්ද?
16. විශාල දත්ත ප්‍රමාණයක් සමඟ nested loops සෑහෙන්න slow වෙන්නේ ඇයි?
17. 2D Array එකක හැම value එකක්ම ලූප් කරන්නේ කොහොමද?
18. Array එකක values විතරක් ඕන නම් තෝරාගන්න ඕන ලූප් එක මොකක්ද?
19. Object එකක keys විතරක් ඕන නම් තෝරාගන්න ඕන ලූප් එක මොකක්ද?
20. අපිට index එකේ පාලනය (control) ඕන නම් තෝරාගන්න ඕන ලූප් එක මොකක්ද?